QUICKSORT: PARADIGMA E ARCHITETTURA

L'ALGORITMO QUICKSORT SFRUTTA IL PARADIGMA DIVIDI ET IMPERA, MA IN MODO OPPOSTO AL MERGE SORT:

- DIVIDI SMART: IL LAVORO PESANTE VIENE SVOLTO PRIMA DELLE CHIAMATE RICORSIVE. L'ARRAY VIENE PARTIZIONATO IN PLACE IN DUE SOTTOARRAY IN BASE A UN ELEMENTO DETTO PIVOT
- COMBINA BANALE: DATO CHE I DUE ARRAY OTTENUTI SONO ORDINATI IN PLACE, IN MODO DA AVERE TUTTI GLI ELEMENTI MINORI DEL PIVOT A SINISTRA E I MAGGIORI A DESTRA, NON OCCORRE ALCUNA OPERAZIONE DI FUSIONE.

PROPRIETÀ DI CORRETTEZZA

AFFINCHÉ QUICKSORT NON VADA IN LOOP INFINITO (CHIAMATE RICORSIVE SU ARRAY DI STESSA DIMENSIONE DEL PADRE), L'INDICE DI DIVISIONE RESTITUITO DALLA FUNZIONE PARTIZIONA DEVE RISPETTARE RIGOROSAMENTE IL VINCOLO $START \leq SPLIT_IDX < END$

```
QUICKSORT(A, START, END)
    IF START < END
        SPLIT_IDX = PARTITION(A, START, END)
        QUICKSORT(A, START, SPLIT_IDX)
        QUICKSORT(A, SPLIT_IDX+1, END)
```

IL CUORE LOGICO: LA PROCEDURA PARTITION

LA PROCEDURA PARTITION RIARRANGIA IL SOTTOARRAY $A[START, \dots, END]$ IN PLACE UTILIZZANDO UN PIVOT. NELL'IMPLEMENTAZIONE VISTA A LEZIONE IL PIVOT È SEMPRE IL PRIMO ELEMENTO DELLA SOTTOSEQUENZA CONSIDERATA ($A[START]$)

INVARIANTE DI PARTIZIONE

AL TERMINE DELL'ESECUZIONE, LA PROCEDURA GARANTISCE LA SEGUENTE PROPRIETÀ DI ORDINAMENTO PARZIALE RISPETTO ALL'INDICE RESTITUITO CURR_J:

- $\forall z \in [START, CURR_J], A[z] \leq PIVOT$
- $\forall t \in [CURR_J+1, END], A[t] \geq PIVOT$

```
PARTITION (A, START, END)
    PIVOT = A[START]
    CURR_I = START - 1
    CURR_J = END + 1
    WHILE TRUE
        CURR_I = CURR_I + 1
        //AVANZA DA SX
        WHILE A[CURR_I] < PIVOT
            CURR_I = CURR_I + 1
        //ARRETRA DA DX
        CURR_J = CURR_J - 1
        WHILE A[CURR_J] > PIVOT
            CURR_J = CURR_J - 1

        IF CURR_I ≥ CURR_J
            RETURN CURR_J
        //SE NON SI INCROCIANO, SCAMBIA E RIPETI
        SWAP(A, CURR_I, CURR_J)
```

ANALISI COMPLESSITÀ

PORCHÉ IL TAGLIO GENERATO DA PARTITION DIPENDE DAI VALORI DELL'ARRAY, L'EQUAZIONE DI RICORRENZA VARIA DRASTICAMENTE IN BASE ALL'INPUT

$$T_{QS}(n) = T_{QS}(q) + T_{QS}(n-q) + \Theta(n)$$

(T_{QS}: QUICKSORT)

A. CASO PEGGIORE: SI VERIFICA QUANDO L'ALGORITMO PRODUCE LO SBILANCIAMENTO MASSIMO POSSIBILE A OGNI INTERAZIONE. PARTITION STACCA UN SOLO ELEMENTO ($q = 1$) E LASCIA TUTTI GLI ALTRI NELL'ALTRA RICORSIONE ($n-1$)

$$T(n) = T(1) + T(n-1) + \Theta(n)$$

LA SOMMA DEI COSTI SU TUTTI I LIVELLI GENERA $\sum_{l=1}^{n} (n-l+1)$, $\Theta(n^2)$

2. CASO MIGLIORE: SE IL PIVOT CADE ESATTAMENTE A METÀ, L'EQUAZIONE DIVENTA $2T(n/2)+\Theta(n)$, IDENTICA A MERGESORT, GENERANDO $\Theta(n \log n)$

CASO MEDIO

PER DIMOSTRARE IL COSTO DEL CASO MEDIO È NECESSARIO SUPPORRE CHE TUTTE LE POSSIBILI PERMUTAZIONI DELL'ARRAY SIANO EQUIPROBABILI POICHÉ IL TEMPO DI ESECUZIONE DIPENDE ESCLUSIVAMENTE DA COME L'ALGORITMO DIVIDE L'ARRAY AD OGNI PASSAGGIO, LA DIMOSTRAZIONE SI BASA SULLA PROBABILITÀ DI OTTENERE I VARI TAGLI.

IL CONCETTO DI RANGO DEL PIVOT

DEFINIAMO RANGO DEL PIVOT $r_A(x)$ COME IL NUMERO DI ELEMENTI DELL'ARRAY CHE SONO MINORI O UGUALI AL PIVOT x. IL RANGO INDICA LA POSIZIONE FINALE IN CUI IL PIVOT DOVREBBE TROVARSI SE L'ARRAY FOSSE GIÀ ORDINATO, IL VALORE DI $r_A(x)$ È COMPRESO TRA 1 E n

LEGAME TRA RANGO E TAGLIO (q)

LA SCELTA DEL PIVOT DETERMINA IL SUO RANGO E IL RANGO DETERMINA UNIVOCAMENTE LA DIMENSIONE DELLA PARTIZIONE SINISTRA (q).

SE IL PIVOT HA RANGO r, CI SONO $r-1$ ELEMENTI PIÙ PICCOLI DI LUI, DI CONSEGUENZA, LA PARTIZIONE SINISTRA AVRÀ DIMENSIONI $q = r-1$ PER $r \geq 2$

L'EQUAZIONE DEL TEMPO MEDIO

POICHÉ L'INPUT È CASUALE ED EQUIPROBABILE, IL PIVOT HA STESSA PROBABILITÀ $\frac{1}{n}$ DI AVERE UN RANGO DA 1 A n. IL TEMPO MEDIO $T_M(n)$ SI CALCOLA SOMMANDO IL COSTO DI PARTIZIONA $\Theta(n)$ E LA MEDIA DI TUTTI I POSSIBILI COSTI DELLE CHIAMATE RICORSIVE PER OGNI POSSIBILE RANGO r:

$$T_M(n) = \frac{1}{n} \sum_{r=1}^{n} (\text{COSTO DELLE RICORSIONI SE IL RANGO È } r)$$

SVILUPPANDO LA SOMMATORIA (DATO CHE OGNI DIMENSIONE q SI PRESENTERÀ STATISTICAMENTE DUE VOLTE, UNA PER IL SOTTOARRAY SINISTRO E UNA PER IL DESTRO), L'EQUAZIONE SI SEMPLIFICA NELLA FORMULA:

$$T_M(n) = \Theta(n) + \frac{2}{n} \sum_{q=1}^{n-1} T_M(q)$$

SOLUZIONE

IPOTIZZANDO CHE LA SOLUZIONE SIA POLILOGARITMICA, OVVERO $T_n(q) \leq c q \log_2 q$,
SI DIMOSTRA CHE SOSTITUENDO QUESTA DISUGUAGLIANZA ALL'INTERNO DELLA SOMMATORIA
E SVOLGENDO I CALCOLI ALGEBRICI, SI DIMOSTRA IL LIMITE SUPERIORE $cn \log_2 n$.
IL COSTO MEDIO RISULTA QUINDI ESSERE $\Theta(n \log_2 n)$